

Revon: A Content-Addressed Versioned Data Store: A Git-Inspired Approach to Structured Data Versioning Using Merkle Hash Tries

[Atharva Sheersh Pandey]¹, [Adhyan Jain]¹, [Poornima Nedunchezian]²

atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in;

poornima.n@vit.ac.in

School of Computer Science and Engineering

VIT Vellore

Abstract

Versioning structured datasets requires durable history, efficient sparse updates, historical reconstruction, and comparison without copying or scanning every record. Revon is a Python prototype built around an immutable fixed-depth Merkle hash trie. A commit rewrites only affected leaf buckets and the union of their ancestor paths; unchanged subtrees are reused by content hash. Revon-H adds addressed changesets and selects between log aggregation and hash-pruned tree differencing using a separately calibrated threshold. We evaluate Snapshot, Log-only, forced-Merkle Revon-M, Revon-H, and Dolt 2.3.1 on identical deterministic workloads up to 100,000 rows. The audited paper run contains 333 successful and correct trials. Under the CLI-facing protocol, Revon-H diff latency was 5.8x to 554.2x lower than Dolt and its incremental commit latency was 2.3x to 37.4x lower, while Revon-H consumed 1.83x to 6.43x more repository storage. Against forced Merkle, the hybrid log path reduced median sparse or hot diff latency by 6.0x to 10.0x. A five-configuration sensitivity study shows that the current $b=8, d=4$ trie minimizes storage in the tested set, whereas $b=8, d=3$ favors latency. The results support a workload-dependent hybrid design, not universal superiority over a production SQL system.

1 Introduction

Version control for structured data is more than archiving files. A useful system must retain exact versions, apply sparse mutations without rebuilding the complete state, reconstruct historical versions, and identify changes efficiently. Git provides the starting idea--immutable objects named by content and commits linked to their parents [7]--but organizes those objects around files and directories. A record store presents a different workload: keys remain stable, update density varies, and two versions may be separat-

ed by a handful of edits or by a long sequence of commits.

Revon evaluates whether a deliberately simple fixed-depth Merkle hash trie can unite incremental structural sharing with an operation index. Its primary variant, Revon-H, uses the log path for short or sparse intervals and switches to the Merkle path when the accumulated operations cross a calibration boundary fixed before evaluation. SQLite provides atomic durable storage, but does not determine the versioning algorithm.

Dolt is the production comparator because it combines SQL semantics, a Git-style commit graph, and content-addressed Prolly Trees [8], [9]. Revon is not presented as a feature replacement for Dolt. The comparison instead identifies the performance and storage trade-offs of a smaller instrumentable mechanism under a reproducible CLI-facing protocol.

1.1 Research Questions

- RQ1: What latency and storage trade-offs does Revon-H exhibit against Snapshot, Log-only, and Dolt across dataset size, mutation density, and locality?
- RQ2: Does adaptive log/Merkle selection improve diff latency relative to a forced-Merkle ablation without changing correctness?
- RQ3: How sensitive are fixed-trie latency, storage, and examined work to branching factor and depth?

1.2 Contributions

- A canonical SHA-256-addressed fixed-depth trie with incremental multi-key path rewriting.
- A hybrid diff policy combining addressed changesets with hash-pruned tree comparison.
- An atomic SQLite repository that persists objects, commits, aliases, metrics, and HEAD and verifies hashes after reopen.

- A five-variant benchmark with shared deterministic workloads, correctness oracles, Dolt integration, raw evidence, and an independently audited summary.
- A controlled branching-factor/depth sensitivity study that prevents treating $b=8$, $d=4$ as universally optimal.

2 Literature Review and Related Work

2.1 Content Addressing and Persistent Structures

Merkle's authenticated tree construction established that a compact root digest can commit to a larger collection [13], while persistent-data-structure theory formalized path copying and bounded-overhead access to historical versions [10]. Git applies content-addressed blobs, trees, and commits to software history [7], and IPFS generalizes content-addressed links into a versionable Merkle DAG [4]. Revon adopts immutable object identity and parent links, but routes structured keys by hash into a persistent trie rather than mirroring a filesystem hierarchy.

Merkle Search Trees combine search-tree ordering with Merkle authentication for efficient state reconciliation [3]. Revon shares hash-pruned comparison but intentionally uses fixed-depth routing and a centralized commit history rather than a replicated CRDT. Content-defined chunking, established in LBFS for redundancy detection [14], also explains why history-independent boundaries are a credible storage improvement beyond the current fixed buckets.

2.2 Structured Dataset Versioning

Dataset-versioning research exposes a broader storage-retrieval design space. DataHub motivates collaborative dataset history at scale [5]. Decibel integrates branching into a relational storage engine and compares version-first, tuple-first, and hybrid layouts [12]. OrpheusDB bolts versioning onto a conventional DBMS and optimizes partitioning for version retrieval [11]. The dataset-versioning model in [6] formalizes the conflict between storing more materialized state and paying more reconstruction cost. Delta Lake represents the complementary log-and-checkpoint design for ACID tables and time travel [1]. These systems motivate reporting commit, checkout, diff, and storage together rather than selecting a single metric.

2.3 System Design Gap

Noms represents structured data as a Merkle DAG of immutable chunks and supports efficient diff and synchronization [2]. ForkBase combines content addressing, fork semantics, and duplicate-content detection for forkable applications [15]. Dolt stores table indexes as content-addressed Prolly Trees: ordered, B-tree-like structures with history-independent chunk boundaries, structural sharing, and native diff [8], [9]. Revon differs by using fixed-depth hash routing and by exposing an explicit alternative operation-log path. This simplifies instrumentation, but gives up Dolt's ordered range behavior, SQL surface, mature branching, merging, and production engineering.

Prior work	Core representation	Relationship to Revon
Persistent DS [10]	Path copying	Formal structural-sharing basis
Git [7]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [4]	Merkle DAG	General content-addressed versioning
MST [3]	Merkle search tree	Authenticated ordered reconciliation
DataHub [5]	Version graph	Collaborative dataset motivation
Decibel [12]	Relational layouts	Native database branching
OrpheusDB [11]	Partitioned versions	Storage/retrieval optimization
Delta Lake [1]	Log/checkpoints	ACID time travel at data-lake scale
Noms [2]	Merkle DAG	Structured content addressing
ForkBase [15]	Forkable CAS	Fork and deduplication semantics
DoltHub [8,9]	Prolly Trees	Production SOTA comparator

Table 1: Literature-review matrix and the gap addressed by Revon.

2.4 Design Implications and Research Positioning

Three separate costs recur in the literature: retained history, creation of a new version, and later reconstruction or comparison. A snapshot keeps reconstruction straightforward by copying unchanged records. A log avoids that copy at commit time, but the deferred work returns during replay, compaction, or checkpointing. Path-copying persistent structures take a third route: only the paths changed by an update are new, while the remaining topology is shared [10]. The

storage-versus-recreation model in [6] makes this trade-off explicit. Decibel and OrpheusDB likewise show that one physical layout does not dominate scans, version retrieval, and storage for every branching workload [11], [12]. For that reason, our evaluation reports import, commit, checkout, diff, and storage rather than using one measurement as a proxy for the system.

Change detection is another point of separation. Comparing two complete states does not need a detailed history, although its work grows with the states being scanned. Replaying changesets instead makes the interval length decisive and requires an unbroken ancestry path with complete changesets. With Merkle differencing, matching subtree hashes stop the search; only mismatching branches are opened. The observed cost then depends on trie shape, leaf occupancy, key distribution, and where updates land. The MST retains key order [3]. Noms and Dolt use content-defined boundaries to form history-independent Prolly Trees [2], [9], while Fork-Base reuses addressed content across objects, branches, and versions [15]. Revon chooses fixed hash-derived routes. This choice removes ordered range access and adaptive boundaries, but gives the experiment a stable geometry whose depth and branching factor can be measured.

The gap addressed here is not a new content-addressed store, but a controlled study in which two correct diff algorithms share one durable history and are chosen from observable properties of the requested interval. Every Revon-H commit records both a Merkle root and an addressed changeset, so changing the diff path does not change the meaning of a version. We expect log aggregation to suit short, sparse intervals and hash pruning to become preferable after many operations. We also expect b and d to change latency, examined work, and storage. Snapshot and Log-only provide representation baselines; Revon-M removes the selector; Revon-H retains it; and Dolt provides a production reference point. This design tests the hybrid mechanism while keeping the system-level Dolt comparison separate from any claim about intrinsic tree superiority.

3 Revon Design

Revon-H architecture

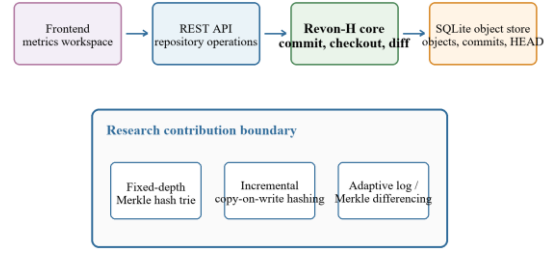


Figure 1: Revon system architecture and the boundary of its research contribution.

3.1 Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON, then names each object with its SHA-256 digest. The commit digest includes the root, parent, changeset, message, and timestamp. Labels such as $v1$ and $v2$ are only human-readable aliases; they do not determine object identity. The durable HEAD reference stores the current commit.

3.2 Fixed-Depth Merkle Hash Trie

For the main experiment we set $b=8$ and $d=4$. Each level reads three bits from $\text{SHA-256}(\text{key})$, so twelve routing bits address 4,096 possible leaf buckets. A bucket still stores the full key, which means that two keys sharing a routing prefix remain distinguishable. Under uniform hashes the expected bucket occupancy is N/b^d . We test other b, d pairs separately instead of assuming that this default is optimal.

3.3 Incremental Hashing and Commit

A mutation batch is canonicalized, routed, and grouped by shared prefixes. Each affected leaf is rewritten once, and only the union of affected ancestor paths is rebuilt. Unchanged child hashes are copied into new internal nodes, providing structural sharing. One SQLite transaction then inserts new objects, the changeset, commit metadata, version alias, measurements, and updated HEAD. Reopen verifies payload hashes and references.

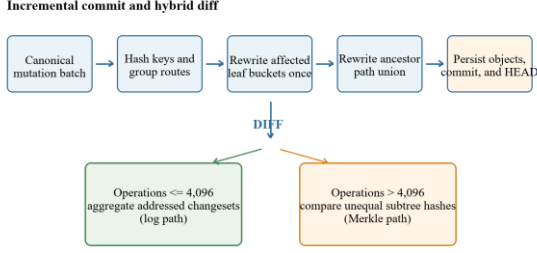


Figure 2: Incremental commit path and adaptive diff decision.

3.4 Hybrid Diff

The log path walks the ancestor chain and aggregates key operations. The Merkle path recursively compares roots, prunes equal subtree hashes, and examines full leaf entries only below unequal paths. Revon-H counts accumulated ancestor operations and uses the log at or below 4,096 operations; it uses Merkle above that boundary. Revon-M forces the Merkle path and therefore serves only as an ablation.

4 Experimental Methodology

4.1 Variants and Workloads

Five implementations receive identical logical states and mutation batches: a complete Snapshot per version, an initial state plus operation-log baseline, forced-Merkle Revon-M, adaptive Revon-H, and Dolt 2.3.1 using a keyed SQL table and native diff. Revon-M is not a headline product variant; it isolates the selector.

Scenario	Rows	Comm.	Changes	Locality
Small sparse	1,000	10	10	spread
Medium sparse	10,000	10	10	spread
Medium dense	10,000	10	1,000	spread
Large sparse	100,000	10	100	spread
Large hot	100,000	10	100	hot

Table 2: Final evaluation workload matrix.

4.2 Measurement Contract

Initial import measures creation of the first durable state; incremental commit is the within-trial median across ten durable commits; diff consumes the complete initial-to-final comparison; checkout fully materializes the target version; storage is the sum of regular repository-file bytes. Correctness requires exact initial and final states plus a changed-key oracle. Work examined uses each system's natural unit and is not normalized across keys, operations, and trie-node pairs.

4.3 Repetition, Calibration, and Integrity

Before measurement, each configuration runs twice without contributing samples. The next seven trials are recorded, using base seed 20260822. For every adapter we regenerate the workload and compare its SHA-256 digest with the manifest before execution. Calibration uses a separate workload set. Reported medians and quartiles exclude warmups. The recorded run identifier is 7048cc1674dc4038a8e42c87818b9088; it was produced on Windows-11-10.0.26200-SP0 with Python 3.14.3. A separate audit rebuilt the trial matrix from the raw CSV and recalculated every summary value.

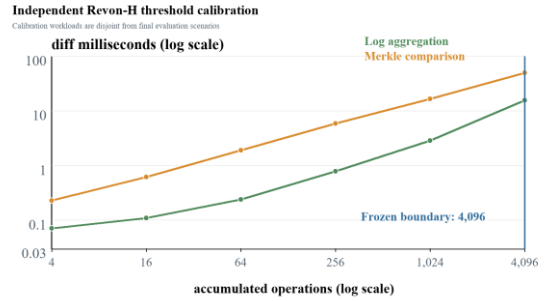


Figure 3: Independent threshold calibration; 4,096 is the largest sampled log-favorable point.

The boundary is a frozen policy for this machine, not proof that the true crossover is exactly 4,096. All 333 paper-profile rows completed with status ok and correctness true. The audit retained all 83 Tukey outlier candidates; no observation was manually deleted.

5 Results

5.1 Final Diff Performance

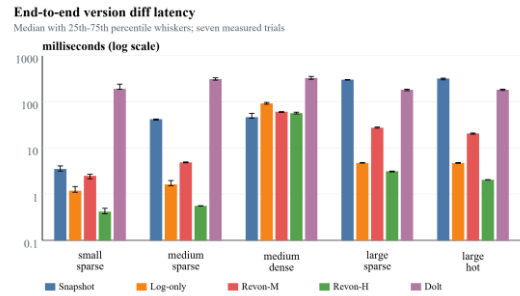


Figure 4: Final diff latency from the audited CSV bundle; lower is better.

Scenario	Path	H ms	Dolt ms	Ratio
Small sparse	log	0.417	188.91	453.3x
Medium sparse	log	0.553	306.47	554.2x
Medium dense	merkle	56.496	326.01	5.8x
Large sparse	log	3.082	180.77	58.7x
Large hot	log	2.031	182.41	89.8x

Table 3: Revon-H versus Dolt median diff latency.

Revon-H selected log for the four sparse or hot workloads and Merkle for the 10,000-operation medium-dense workload. Relative to Dolt, its median diff was 5.8x lower in medium-dense and 58.7x to 554.2x lower in the other workloads. These ratios include repeated Dolt CLI and SQL invocation costs and therefore describe the evaluated system protocol, not isolated Prolly Tree algorithm speed.

5.2 Hybrid Ablation

For log-selected workloads, Revon-H reduced median diff latency relative to forced-Merkle Revon-M by 6.0x (small sparse), 8.7x (medium sparse), 8.8x (large sparse), and 10.0x (large hot). In medium-dense, both used Merkle and Revon-H was within 7.3% of the forced variant. This supports the selector's intended mechanism without claiming that its threshold is hardware-independent.

5.3 Commit, Checkout, and Storage

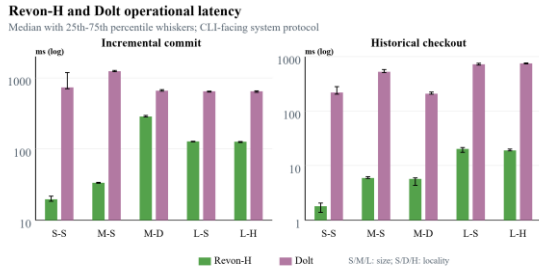


Figure 5: Revon-H and Dolt operational latency; lower is better.

Revon-H incremental commit medians were 2.3x to 37.4x lower than Dolt, and checkout medians were 35.8x to 123.4x lower. Initial import was mixed: Revon-H was 4.9x faster for small sparse and 1.14x faster for medium sparse, approximately tied for medium dense, and 2.73x to 2.79x slower at 100,000 rows. The latency advantage is therefore not uniform.

Revon-H paid a consistent storage cost. Its median repository footprint was 1.83x, 3.16x, 2.19x, 4.10x, and 6.43x Dolt's for small sparse through large hot, respectively. The 100,000-row

Revon repositories occupied about 34.4 MiB. This reflects inline values, JSON object encoding, and the absence of compact chunk packing and garbage collection.

5.4 Trie-Parameter Sensitivity

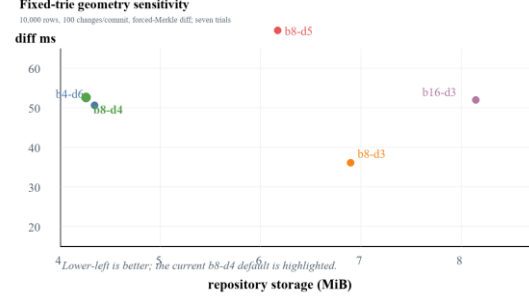


Figure 6: Fixed-trie sensitivity; the preferred point depends on the objective.

All 45 sensitivity trials were correct. The b8-d3 configuration delivered the lowest median diff latency (20.15 ms versus 44.43 ms for b8-d4) and compared 84.3% fewer node pairs, but consumed 61.2% more storage and examined three times as many leaf entries. The current b8-d4 default used the least storage (4.27 MiB). The over-deep b8-d5 design was slower for every timed operation and used 44.8% more storage. Thus b8-d4 is a storage-conscious balanced default, not a universal optimum.

6 Discussion

The results answer RQ1 with a trade-off rather than a single winner. Revon-H offered low diff, commit, and checkout latency under the benchmark interface, but its import advantage disappeared at 100,000 rows and its repository was always larger than Dolt's. Snapshot and Log-only also remain relevant baselines: their simpler representations often reduced import or storage cost. Snapshot comparison, however, grew with the number of materialized keys.

RQ2 is supported more directly. Because Revon-H and Revon-M share the same durable trie, their sparse-workload diff gap isolates adaptive selection more cleanly than an absolute comparison with Dolt. The log path avoided thousands of trie comparisons when only 100 or 1,000 operations separated the endpoints; the dense case switched to Merkle.

RQ3 rejects a universal geometry. Shallow routing lowers internal traversal but creates larger leaf buckets and higher storage; deeper rout-

ing lowers occupancy but increases internal-node work. The suitable configuration therefore depends on which requirement dominates the workload: latency, storage, or worst-case leaf-bucket scanning.

The Dolt ratios need a system-level interpretation. Dolt performs SQL parsing, schema management, version-control operations, and production durability work that the Revon prototype does not. Dolt's Prolly Trees also serve ordered queries, branches, merges, and remote workflows. Our timings compare the declared interfaces used in the experiment. They do not isolate the tree algorithms and therefore cannot show that a fixed trie is inherently faster than a Prolly Tree.

7 Limitations and Threats to Validity

- Runtime and interface asymmetry: Revon executes in-process in Python, whereas Dolt is a mature Go executable invoked repeatedly through CLI and SQL interfaces.
- Feature asymmetry: Revon supports one writer and a linear history, but not SQL, branches, merges, remotes, schema evolution, or production concurrency.
- Synthetic scope: the deterministic key/value workloads reach 100,000 rows, but cannot represent every real schema, key skew, payload size, or history length.
- Single environment: all results were obtained on one Windows 11 machine, where cache state, scheduling, filesystem behavior, and antivirus activity can affect short measurements.
- Calibration boundary: 4,096 is the largest sampled operation count that favored the log; this does not establish a universal crossover.
- Memory comparability: Python tracemalloc and Dolt process-memory readings are not equivalent, so no cross-system memory graph is reported.
- Work-unit mismatch: keys, log operations, trie-node pairs, and Dolt's internal work do not form a single normalized measure.
- Outlier policy: all 83 Tukey candidates were kept. Medians and interquartile ranges limit their influence, but seven measured trials do not demonstrate normality.

8 Future Work

A next version should choose its structure from observed workload statistics. Key count, prefix skew, mutation density, and leaf occupancy could guide the choice among measured trie geometries instead of fixing b and d for every repository. The diff selector could recalibrate periodically and consider history distance, changed-prefix density, and observed I/O cost alongside its present operation-count rule.

Storage efficiency could be improved with binary node records, separate value chunks, packed objects, compression, reachability checks, and garbage collection. An ordered Merkle index or content-defined chunk boundaries could add range scans and a history-independent layout while leaving the fixed trie as the controlled baseline.

To move beyond a linear prototype, Revon still needs named branches, merge commits, conflict reporting, schema-aware records, concurrent readers and writers, and remote synchronization. Evaluation should add public structured datasets, one-million-row and longer-history workloads, Linux replication, and unified external process telemetry for time, CPU, I/O, and peak resident memory.

9 Conclusion

Revon demonstrates a complete content-addressed versioning path for structured key/value data: canonical immutable objects, incremental copy-on-write Merkle trie updates, atomic SQLite persistence, historical checkout, and adaptive differencing. The audited 333-trial comparison shows that Revon-H can substantially reduce CLI-facing diff, commit, and checkout latency relative to Dolt 2.3.1, but it also exposes slower large imports, higher storage consumption, a narrower feature set, and runtime/interface asymmetry. The ablation supports hybrid log/Merkle selection, while the sensitivity experiment shows that trie geometry must be chosen by workload objective. The defensible contribution is therefore an instrumentable hybrid design and reproducible evidence of its trade-offs, not a claim of universal replacement for a production version-controlled SQL database.

Acknowledgment

The authors acknowledge the open-source Dolt project and the researchers whose dataset-versioning systems informed the experimental design.

References

- [1] Michael Armbrust et al. 2020. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12):3411-3424. doi:10.14778/3415478.3415560.
- [2] Attic Labs. n.d. Noms technical overview. GitHub. <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [3] Alex Auvolat and Francois Taiani. 2019. Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. 38th IEEE International Symposium on Reliable Distributed Systems, pages 1-10. doi:10.1109/SRDS47363.2019.00032.
- [4] Juan Benet. 2014. IPFS - Content Addressed, Versioned, P2P File System. arXiv:1407.3561.
- [5] Anant Bhardwaj, Souvik Bhattacharjee, Amit Chavan, Amol Deshpande, Aaron J. Elmore, Samuel Madden, and Aditya G. Parameswaran. 2015. DataHub: Collaborative Data Science and Dataset Version Management at Scale. 7th Biennial Conference on Innovative Data Systems Research.
- [6] Souvik Bhattacharjee, Amit Chavan, Silu Huang, Amol Deshpande, and Aditya Parameswaran. 2015. Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff. *Proceedings of the VLDB Endowment*, 8(12):1346-1357. doi:10.14778/2824032.2824035.
- [7] Scott Chacon and Ben Straub. 2014. Git Internals - Git Objects. In *Pro Git*, 2nd edition. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [8] DoltHub. 2024a. Dolt Storage Engine: Block Store. Dolt Documentation. <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [9] DoltHub. 2024b. Dolt's Storage Engine: Prolly Trees and Commit Graph. <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- [10] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. 1989. Making Data Structures Persistent. *Journal of Computer and System Sciences*, 38(1):86-124. doi:10.1016/0022-0000(89)90034-2.
- [11] Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proceedings of the VLDB Endowment*, 10(10):1130-1141. doi:10.14778/3115404.3115417.
- [12] Michael A. Maddox et al. 2016. Decibel: The Relational Dataset Branching System. *Proceedings of the VLDB Endowment*, 9(9):624-635. doi:10.14778/2947618.2947619.
- [13] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology - CRYPTO '87*, LNCS 293, pages 369-378. doi:10.1007/3-540-48184-2_32.
- [14] Athicha Muthitacharoen, Benjie Chen, and David Mazieres. 2001. A Low-Bandwidth Network File System. *Proceedings of the 18th ACM Symposium on Operating Systems Principles*, pages 174-187. doi:10.1145/502034.502052.
- [15] Sheng Wang et al. 2018. ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications. *Proceedings of the VLDB Endowment*, 11(10):1137-1150. doi:10.14778/3231751.3231762.

Appendix A Reproducibility and Data Availability

The repository contains the benchmark harness, evidence audit, trie-sensitivity harness, and paper generator. Source code, artifacts, and reproduction instructions are available at <https://github.com/atharvasheersh/Revon>. The final evidence bundle is evidence/paper-final-20260824; the sensitivity bundle is evidence/trie-sensitivity-20260824. Results in this paper are generated programmatically from their summary CSV files and verified against raw results and manifests; no table or graph uses an earlier validation export.